

EduSync 阶段学习总结

Google 一键登录 + 全栈部署实战知识手册

项目: EduSync (React/Vite + Flask + Supabase)

日期: 2026年6月10日

前端: <https://edu-sync-gamma.vercel.app>

后端: <https://edusync-production-6d33.up.railway.app/api>

用途: 复盘今日 OAuth / 部署踩坑, 作为下次上线的检查清单

目录

1. 今天完成了什么
2. 全栈架构一张图
3. OAuth 2.0 与 Google 登录（核心）
4. Supabase Auth 配置知识点
5. 前端实现知识点（Vite + React）
6. 后端实现知识点（Flask + Railway）
7. 环境变量：最容易踩坑的地方
8. 今天遇到的所有错误 → 原因 → 解法
9. 部署与 DevOps 知识点
10. 安全与密钥管理
11. 调试方法论（你今天实际用到的）
12. 值得深入学习的技能路线图
13. 术语表（中英对照）
14. 检查清单（下次上线直接照着做）

Google 一键登录 + 全栈部署实战知识手册

项目：EduSync (React/Vite 前端 + Flask 后端 + Supabase)

日期：2026年6月10日

作者学习场景：独立完成 Google OAuth、Vercel/Railway 上线、跨平台排错

线上地址：

- 前端： <https://edu-sync-gamma.vercel.app>
 - 后端： <https://edusync-production-6d33.up.railway.app/api>
 - Supabase： <https://ptxrmujnqrvwakfpdyhh.supabase.co>
-

目录

1. 今天完成了什么
 2. 全栈架构一张图
 3. OAuth 2.0 与 Google 登录 (核心)
 4. Supabase Auth 配置知识点
 5. 前端实现知识点 (Vite + React)
 6. 后端实现知识点 (Flask + Railway)
 7. 环境变量：最容易踩坑的地方
 8. 今天遇到的所有错误 → 原因 → 解法
 9. 部署与 DevOps 知识点
 10. 安全与密钥管理
 11. 调试方法论 (你今天实际用到的)
 12. 值得深入学习的技能路线图
 13. 术语表 (中英对照)
 14. 检查清单 (下次上线直接照着做)
-

1. 今天完成了什么

1.1 功能层面

完成项	说明
Google 一键登录按钮	登录页、注册页均可使用
OAuth 回调页 /auth/callback	处理 Google 返回的 token
首次 Google 用户选角色	Teacher / Student 弹窗（ OAuthRoleDialog ）
老用户直接登录	跳过角色选择，进 Dashboard
后端 OAuth 接口	POST /api/auth/oauth/complete 与 POST /api/auth/oauth/register
生产环境全链路跑通	Vercel 前端 + Railway 后端 + Supabase + Google Cloud

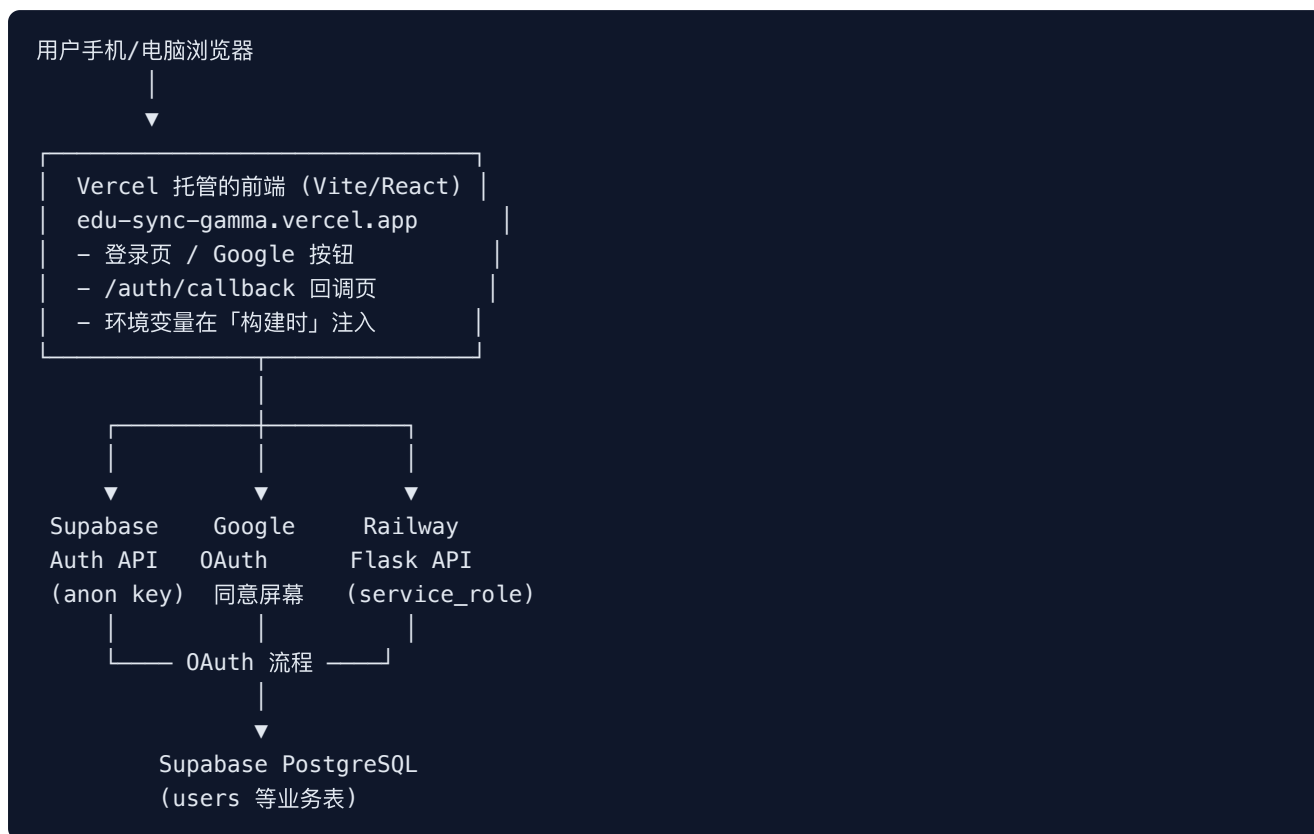
1.2 工程层面

完成项	说明
Vercel 环境变量配置	VITE_API_URL 、 VITE_SUPABASE_URL 、 VITE_SUPABASE_ANON_KEY
Railway 后端重新部署	含 OAuth 路由的最新代码
Python 构建问题修复	runtime.txt 、 mise.toml 、 nixpacks.toml 等
多种 OAuth 错误排查	redirect、PKCE、Failed to fetch、404、构建失败

1.3 你现在的项目阶段（结合 MVP-PLAN）

- **Week 3 目标 (Auth)**：基本完成，且超出计划（多了 Google OAuth）
 - **Week 4（业务 API 对接）**：可以正式开始（登录体系已稳定）
 - **尚未完成**：自定义域名、SEO 收录、邮件登录、完整 E2E 测试文档等
-

2. 全栈架构一张图



关键理解：

- 前端只负责 UI + 发起 OAuth + 拿 Supabase token + 调你自己的后端
- Supabase负责身份验证（Google 对接、发 token）
- 你的 Flask 后端负责业务逻辑（查 `users` 表、发你自己的 JWT、注册角色）
- 三个平台、两套密钥、两种 redirect URL——搞混就会报错

3. OAuth 2.0 与 Google 登录（核心）

3.1 OAuth 是什么？（用人话）

你想让用户用 Google 账号登录你的 App，但你不能要用户的 Google 密码。

OAuth 就是：用户去 Google 说「我同意」，Google 给 Supabase 一个临时通行证（code/token），Supabase 再转给你的网站。

3.2 你的项目里实际发生的两次跳转

第一次 redirect（Google ↔ Supabase）：

Google 授权后 → <https://ptxrmujnqrvwakfpdyhh.supabase.co/auth/v1/callback>

- 这个地址填在 **Google Cloud Console** → OAuth 客户端 → 已授权的重定向 URI
- 不要填你的 Vercel 地址

第二次 redirect (Supabase ↔ 你的网站):

```
Supabase 处理后 → https://edu-sync-gamma.vercel.app/auth/callback
```

- 这个地址填在 **Supabase** → URL Configuration → **Redirect URLs**
- 代码里 `redirectTo: ${window.location.origin}/auth/callback`

3.3 PKCE vs Implicit (你今天踩坑的重点)

对比项	PKCE 流程	Implicit 流程
返回形式	URL 带 <code>?code=xxx</code>	URL 带 <code>#access_token=xxx</code>
安全性	更高 (推荐生产)	较低, 但 SPA 简单
前端要求	必须存 <code>code_verifier</code>	不需要 verifier
你今天的问题	verifier 丢失 → 报错	最终采用此方案解决
适用场景	Next.js SSR、有 cookie	纯 Vite SPA、一次性 token

你学到的结论:
纯前端 SPA (无服务端存 cookie) 时, PKCE 容易在跨站跳转后丢 verifier; Implicit 更省事, 你们只用 token 换自己的 JWT, 可接受。

3.4 Google Cloud Console 必会配置

1. 项目 → APIs & Services → **OAuth 同意屏幕** (测试模式要加测试用户)
2. 凭据 → 创建 **OAuth 2.0 客户端 ID** → 类型选 **Web 应用**
3. 已授权的重定向 URI (仅 Supabase):
`https://ptxrmujnqrwvafpdyyh.supabase.co/auth/v1/callback`
4. 复制 **Client ID** (`.apps.googleusercontent.com`) 和 **Client Secret** (`GOCSPX-` 开头)
5. **Secret 必须从网页复制, 不要从截图 OCR**——错一个字符就 `Unable to exchange external code`

3.5 值得学习的 OAuth 延伸知识

- [] OAuth 2.0 四种授权模式 (授权码、隐式、客户端、PKCE)
- [] `state` 参数防 CSRF
- [] OpenID Connect (OIDC) 与 `id_token` 区别
- [] 为什么生产环境更推荐 PKCE + 服务端换 token (Next.js 模式)

4. Supabase Auth 配置知识点

4.1 两个 URL 配置（最容易混）

配置项	填什么	不要填什么
Site URL	<code>https://edu-sync-gamma.vercel.app</code>	不要带 <code>/auth/callback</code>
Redirect URLs	<code>https://edu-sync-gamma.vercel.app/auth/callback</code>	必须有 <code>https://</code>

4.2 两种 API Key（必须分清）

密钥	放在哪	用途	能否暴露给浏览器
anon key	Vercel <code>VITE_SUPABASE_ANON_KEY</code>	前端发起 OAuth	可以（公开）
service_role key	Railway <code>SUPABASE_SERVICE_ROLE_KEY</code>	后端读写数据库	绝对不行

4.3 Supabase 在 OAuth 中的角色

1. 前端 `signInWithOAuth({ provider: 'google' })` → 跳 Supabase
2. Supabase 跳 Google
3. Google 回 Supabase `/auth/v1/callback`
4. Supabase 验证后跳你的 `/auth/callback`，带上 token
5. 你的前端拿 `access_token` 调自己后端 `/api/auth/oauth/complete`

4.4 值得学习的 Supabase 延伸

- [] Row Level Security (RLS) 策略
 - [] Supabase Auth Hooks
 - [] 用 Supabase 只做 Auth，业务用户表自己维护（你现在的模式）
 - [] `@supabase/ssr` 在 Next.js 里的用法
-

5. 前端实现知识点 (Vite + React)

5.1 关键文件与职责

文件	职责
src/lib/supabase.ts	创建 Supabase 客户端, flowType: 'implicit'
src/lib/googleAuth.ts	startGoogleSignIn(), 设置 redirectTo
src/pages/AuthCallbackPage.tsx	解析 hash/query, 调后端完成登录
src/components/OAuthRoleDialog.tsx	首次用户选 Teacher/Student
src/lib/api.ts	completeOAuthSignIn()、registerOAuthUser()
src/App.tsx	路由 /auth/callback

5.2 SPA 路由 vs 真实文件

- 没有 callback/ 文件夹
- /auth/callback 是 React Router 的客户端路由
- Vercel 需要 vercel.json 把所有路径 fallback 到 index.html (SPA 标配)

5.3 Vite 环境变量规则

- 必须以 VITE_ 开头才会打进前端包
- 在 构建时 注入, 不是运行时
- 改 Vercel 环境变量后必须 Redeploy, 否则旧包还在用旧值

5.4 值得学习的前端延伸

- [] React Router 保护路由 (ProtectedRoute)
- [] AuthContext 与 token 持久化 (localStorage)
- [] 如何用浏览器 DevTools → Network 看 OAuth 请求链
- [] TypeScript 类型定义 API 响应 (OAuthCompleteResponse)

6. 后端实现知识点 (Flask + Railway)

6.1 OAuth 相关接口

POST /api/auth/oauth/complete

- 入参: { "access_token": "..."} (Supabase 发的 token)

- 用 `supabase.auth.get_user(token)` 验证用户
- 若 `users` 表有记录 → 返回 `{ status: "ok", token, user }`
- 若没有 → 返回 `{ status: "needs_profile", email, suggested_display_name }`

POST /api/auth/oauth/register

- 首次 Google 用户提交 `role + display_name`
- 写入 `users` 表

6.2 为什么需要自己的后端？（不能只靠 Supabase）

- 你的业务用 `users` 表的 `role` (teacher/student)
- 你的 JWT / 会话逻辑在 Flask
- Google 只证明「这是哪个 Google 账号」，不证明「在 EduSync 里他是老师还是学生」

6.3 CORS

```
CORS(app) # 允许前端跨域访问 API
```

- 前端在 `vercel.app`，后端在 `railway.app` → 跨域
- 没有 CORS 浏览器会报 `Failed to fetch`（有时）

6.4 值得学习的后端延伸

- [] Flask Blueprint 组织路由
 - [] `require_auth` 装饰器与 JWT 校验
 - [] Gunicorn + Procfile 生产部署
 - [] API 错误码设计 (400/401/404/500)
-

7. 环境变量：最容易踩坑的地方

7.1 三份环境变量对照表

变量	本地 .env	Vercel	Railway
VITE_API_URL	可选	✅ 必须	❌
VITE_SUPABASE_URL	✅	✅	❌
VITE_SUPABASE_ANON_KEY	✅	✅	❌
SUPABASE_URL	✅	❌	✅
SUPABASE_SERVICE_ROLE_KEY	✅	❌	✅
FLASK_ENV	development	❌	production
FRONTEND_URL	localhost	❌	你的 Vercel 地址
MISE_PYTHON_GITHUB_ATTESTATIONS	❌	❌	false (构建用)

7.2 记忆口诀

- VITE_ 开头 = 给浏览器用的，在 Vercel 配
 - SUPABASE_SERVICE_ROLE = 给服务器用的，只在 Railway 配
 - 改完 Vercel 变量 = 必须 Redeploy
 - 改完 Railway 变量 = 通常自动重新部署
-

8. 今天遇到的所有错误 → 原因 → 解法

8.1 错误速查表

错误信息	根因	解法
Google 按钮灰色/不可用	缺 <code>VITE_SUPABASE_*</code>	根目录 <code>.env</code> + Vercel 环境变量 + Redeploy
<code>provider is not enabled</code>	Supabase 未开 Google	Providers → Google → Enable
<code>requested path is invalid</code>	Redirect URL 不在白名单	Supabase 加完整 <code>https://.../auth/callback</code>
<code>Unable to exchange external code</code>	Google Secret 错 / Google redirect URI 错	重抄 Secret; Google 只填 Supabase callback
<code>No Google session found</code>	回调页读不到 token	改 implicit 流程 + <code>detectSessionInUrl</code>
<code>PKCE code verifier not found</code>	PKCE verifier 跨跳转丢失	改用 implicit flow
<code>Failed to fetch</code>	后端未部署 / CORS / API URL 错	查 Railway; 确认 <code>VITE_API_URL</code>
<code>curl</code> 返回 404	Railway 跑旧代码	重新部署后端
Railway <code>python@3.13.14</code> 失败	Python 太新无预编译包	<code>runtime.txt</code> → 3.12.8
Railway attestations 失败	mise 安全校验	<code>mise.toml</code> + <code>MISE_PYTHON_GITHUB_ATTESTATIONS=false</code>
<code>Authentication failed</code> (curl test)	假 token 预期行为	✅ 说明接口已上线

8.2 诊断命令（以后直接用）

```
# 1. 后端健康
curl https://edusync-production-6d33.up.railway.app/api/health

# 2. OAuth 接口是否存在（应 401, 不是 404）
curl -X POST "https://edusync-production-6d33.up.railway.app/api/auth/oauth/complete" \
  -H "Content-Type: application/json" \
  -d '{"access_token":"test"}'

# 3. 普通登录接口（对照）
curl -X POST "https://edusync-production-6d33.up.railway.app/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"x"}'
```

8.3 看地址栏判断卡在哪一步

地址栏特征	卡在哪
supabase.co + error_description	Google ↔ Supabase 配置问题
/auth/callback?code=	PKCE 阶段（旧流程）
/auth/callback#access_token=	Implicit 成功返回
/auth/callback 无参数	刷新太早或流程中断
页面 Failed to fetch	前端连不上 Railway

9. 部署与 DevOps 知识点

9.1 Vercel（前端）

- 从 GitHub main 自动部署
- Root Directory: 仓库根目录
- Build: npm run build → 输出 dist/
- 环境变量在 build 时打进 JS

9.2 Railway（后端）

- Root Directory 必须是 backend
- Start Command: gunicorn run:app --bind 0.0.0.0:\$PORT （见 Procfile）
- 环境变量在 运行时 读取
- 部署失败时，旧版本继续服务——所以你会看到 login 200 但 oauth 404

9.3 Git 与仓库迁移

你 push 时看到：

```
remote: This repository moved. Please use:
remote:  https://github.com/ZacharyZhou/EduSync.git
```

- 旧 URL 可能仍可用，但 Railway 应确认连的是正确仓库
- 部署触发依赖 GitHub webhook

9.4 Python 构建文件（你今天新增的）

文件	作用
backend/runtime.txt	指定 python-3.12.8
backend/.python-version	mise 读版本
backend/nixpacks.toml	Nixpacks 构建变量
backend/mise.toml	关闭 Python attestations

9.5 值得学习的 DevOps 延伸

- [] GitHub Actions CI/CD（自动测试再部署）
- [] 自定义域名 + HTTPS（Vercel Domains）
- [] Railway / Vercel 日志与监控
- [] Staging 环境（preview 分支自动部署）
- [] Docker 容器化（绕过 mise 问题）

10. 安全与密钥管理

10.1 今天遵守的好习惯

-  service_role 只在 Railway，不进 Git
-  .env 在 .gitignore
-  前端只用 anon key
-  OAuth 完成后 supabase.auth.signOut()，用自己 JWT

10.2 绝对不要做的事

-  把 Client Secret 截图 OCR 复制
-  把 SUPABASE_SERVICE_ROLE_KEY 写进 Vercel

- ❌ 把 `.env` commit 到 GitHub
- ❌ 在 Google Cloud 填 Vercel 地址作 redirect

10.3 值得学习的安全延伸

- [] JWT 过期与刷新策略
- [] HTTPS 全链路
- [] OWASP Top 10 基础
- [] Supabase RLS 防止越权读数据

11. 调试方法论（你今天实际用到的）

11.1 分层排查法

```
第 1 层: UI 能不能点? → 环境变量 / Supabase 客户端
第 2 层: Google 能不能授权? → Google Cloud + Supabase Provider
第 3 层: 能不能回到 callback? → Redirect URLs
第 4 层: callback 有没有 token? → OAuth 流程 PKCE/Implicit
第 5 层: 后端能不能接? → Railway 部署 / VITE_API_URL
第 6 层: 业务能不能完成? → users 表 / register 接口
```

不要跳层。你今天多次在 Layer 5 出问题，但 Layer 2–4 已修好。

11.2 工具箱

工具	用途
浏览器无痕窗口	排除缓存干扰
地址栏 URL	看 <code>code</code> / <code>access_token</code> / <code>error</code>
<code>curl</code>	绕开浏览器测 API
Chrome DevTools → Network	看 Failed to fetch 打哪
Railway Deploy Logs	构建/启动失败
Vercel Deployments	前端是否最新 build

11.3 心态

- 404 vs 401 差一个数字，意义完全不同（不存在 vs 存在但鉴权失败）
- 构建失败 ≠ 网站挂了，可能是旧版本还在跑
- 错误信息变了吗 = 你在往前推进

12. 值得深入学习的技能路线图

12.1 优先级 P0（马上有用，和你项目直接相关）

1. OAuth 2.0 + Supabase Auth 官方文档走一遍
2. HTTP 基础：状态码、CORS、JSON API
3. 环境变量与构建：Vite vs Flask 差异
4. Git 基础：commit、push、看 log
5. curl 与 Postman：独立测 API

12.2 优先级 P1（1–2 周内）

1. React Router + Context *auth 模式*
2. Flask Blueprint + 中间件
3. PostgreSQL / Supabase SQL
4. Railway + Vercel 文档各读一章
5. 浏览器 DevTools Network 面板

12.3 优先级 P2（进阶）

1. TypeScript 类型系统
2. PKCE + Next.js SSR 正确做法
3. CI/CD (GitHub Actions)
4. 自定义域名与 DNS
5. 基础 SEO (Search Console)

12.4 推荐学习资源

主题	资源
OAuth	Supabase Google Auth 文档
部署	项目内 <code>DEPLOY.md</code>
MVP 进度	项目内 <code>MVP-PLAN.md</code>
PKCE	OAuth 2.0 PKCE RFC 7636

13. 术语表（中英对照）

中文	English	一句话解释
授权码	Authorization Code	Google 给 Supabase 的临时码
访问令牌	Access Token	证明用户身份的 JWT
回调地址	Redirect URI	OAuth 完成后跳回的 URL
跨域	CORS	浏览器限制不同域名间的请求
匿名密钥	anon key	前端可用的 Supabase 公钥
服务角色密钥	service_role key	后端超级权限密钥
隐式流程	Implicit Flow	token 直接放 URL hash
PKCE	Proof Key for Code Exchange	用 verifier 增强安全的授权码模式
构建时注入	Build-time env	Vite 打包时写死变量
单页应用	SPA	一个 HTML，路由靠 JS
蓝图	Blueprint	Flask 模块化路由
部署	Deploy	把代码发布到公网服务器
健康检查	Health Check	<code>/api/health</code> 看服务是否活着

14. 检查清单（下次上线直接照着做）

14.1 Google 登录上线前

- [] Google Cloud: Web 应用 OAuth 客户端
- [] Google redirect: `https://<project>.supabase.co/auth/v1/callback`
- [] Supabase Google Provider: ID + Secret 无空格
- [] Supabase Site URL: 前端域名（无路径）
- [] Supabase Redirect URLs: 前端 `/auth/callback`（含 https）
- [] Vercel 三个 `VITE_*` 变量 + Redeploy
- [] Railway `SUPABASE_*` + Root Directory = `backend`
- [] `curl` `oauth/completed` 返回 401（非 404）
- [] 无痕窗口完整走一遍 Google 登录

14.2 每次改代码后

- [] `git push origin main`
- [] Vercel Deployment Success (前端)
- [] Railway Deployment Success (后端)
- [] 用手机测一次 (响应式 + 登录)

结语

你今天完成的不只是「加一个 Google 按钮」，而是一条完整的**现代 Web 全栈链路**：

第三方身份 (Google) → 身份平台 (Supabase) → 自己的业务后端 (Flask) → 自己的数据库 (PostgreSQL) → 托管在两家云平台 (Vercel + Railway)

能独立把这条链路跑通，已经远超「只会写页面」的阶段。

今天踩的每一个坑 (redirect、PKCE、Failed to fetch、Python 构建) 都是真实公司里会遇到的部署问题。

保存这份 PDF，下次做 Apple 登录、GitHub 登录、或换域名时，80% 的步骤可以复用。

EduSync Learning Guide · Generated 2026-06-10 · Zachary Zhou

EduSync Learning Guide · Generated 2026-06-10 · 保存此 PDF 供 Apple/GitHub 登录与换域名时复用